

## Git and the lab notebook

---

- A **lab notebook** records what you tried, what worked, and what failed.

- A **lab notebook** records what you tried, what worked, and what failed.
- **Computational** work lives in **code**; a separate notebook duplicates that story.

- A **lab notebook** records what you tried, what worked, and what failed.
- **Computational** work lives in **code**; a separate notebook duplicates that story.
- **Version control** ties narrative to files: saves carry messages; you can **compare** and **roll back**.

- More than one version-control system exists; **git** is the norm in science and engineering.

- More than one version-control system exists; **git** is the norm in science and engineering.
- **GitHub** (and similar hosts) provide storage, sharing, and review on top of git.

- Bash basics

## In this module

- Bash basics
- Advanced bash

## In this module

- Bash basics
- Advanced bash
- Bash scripting

## In this module

- Bash basics
- Advanced bash
- Bash scripting
- SSH Keys

## In this module

- Bash basics
- Advanced bash
- Bash scripting
- SSH Keys
- Branching and workflows

## In this module

- Bash basics
- Advanced bash
- Bash scripting
- SSH Keys
- Branching and workflows
- File organization

## In this module

- Bash basics
- Advanced bash
- Bash scripting
- SSH Keys
- Branching and workflows
- File organization
- Makefile and justfile

## Why use the command line?

- GUIs for git are fine for many tasks.

## Why use the command line?

- GUIs for git are fine for many tasks.
- The **CLI** behaves the same on a laptop, in a container, and on a cluster login node.

## Why use the command line?

- GUIs for git are fine for many tasks.
- The **CLI** behaves the same on a laptop, in a container, and on a cluster login node.
- Everyone hits the same commands and error messages when asking for help.

- `git --help` lists subcommands.

- `git --help` lists subcommands.
- `git <command> --help` documents one command in full.

- `git --help` lists subcommands.
- `git <command> --help` documents one command in full.
- Learn a **small** set by heart; use help and GUIs for the rest.

## What version control records

- An annotated history of every tracked file you commit.

## What version control records

- An **annotated history** of every **tracked** file you commit.
- Edits in your working tree are **not** history until you **commit**.

## What version control records

- An **annotated history** of every **tracked** file you commit.
- Edits in your working tree are **not** history until you **commit**.
- Each **commit** stores a snapshot plus metadata (time, author, parents, etc.).

## Tell git who you are

```
git config --global user.email "you@example.com"  
git config --global user.name "Your Name"  
git config --global --list
```

## Tell git who you are

```
git config --global user.email "you@example.com"  
git config --global user.name "Your Name"  
git config --global --list
```

- Run once per machine (or container). Commits are stamped with this identity.

1. Edit files.

## The usual loop

1. Edit files.
2. Stage with `git add` (what enters the next snapshot).

## The usual loop

1. Edit files.
2. Stage with `git add` (what enters the next snapshot).
3. Commit with `git commit` and a message that states **why** the change matters.

## The usual loop

1. **Edit** files.
2. **Stage** with `git add` (what enters the next snapshot).
3. **Commit** with `git commit` and a message that states **why** the change matters.
4. (Optional) **Push** to GitHub to save the commit to the remote repository.

- Use an empty directory (e.g. under `/workspace` in the class container).

- Use an empty directory (e.g. under `/workspace` in the class container).

- Use an empty directory (e.g. under `/workspace` in the class container).

```
mkdir -p git-practice && cd git-practice  
git init  
echo "test file" > test_file.txt  
git status
```

- Use an empty directory (e.g. under `/workspace` in the class container).

```
mkdir -p git-practice && cd git-practice
git init
echo "test file" > test_file.txt
git status
```

- Untracked files appear in `git status` until you add them.

```
git add test_file.txt  
git status
```

```
git add test_file.txt
```

```
git status
```

- Staging marks the file for the next commit; it is not yet permanent history.

## Make the first commit

```
git commit -m "initial add"  
git status
```

## Make the first commit

```
git commit -m "initial add"
```

```
git status
```

- Clean working tree: files on disk match the latest commit.

- Every commit gets a unique **hash** (content + metadata).

- Every commit gets a unique **hash** (content + metadata).
- Refer to commits by hash; the first several characters often suffice.

- Every commit gets a unique **hash** (content + metadata).
- Refer to commits by hash; the first several characters often suffice.
- Hashes identify snapshots unambiguously when you merge, revert, or compare.

```
git log
```

`git log`

- Newest commits usually appear **first**.

### `git log`

- Newest commits usually appear **first**.
- Each line ties a **hash** to a **message**—your notebook entry for that step.

- Push to GitHub to browse history in a web UI.

- **Push** to GitHub to browse history in a web UI.
- **VS Code** and **Cursor** expose **Source Control** for stage/commit/push flows.

- **Push** to GitHub to browse history in a web UI.
- **VS Code** and **Cursor** expose **Source Control** for stage/commit/push flows.
- CLI, web, and editor are views on the **same** commits.

## What changed? `git diff`

- Inspect unstaged edits before you add:

## What changed? `git diff`

- Inspect unstaged edits before you add:

## What changed? `git diff`

- Inspect unstaged edits before you add:

```
echo "another line" >> test_file.txt  
git diff test_file.txt
```

## What changed? `git diff`

- Inspect unstaged edits before you add:

```
echo "another line" >> test_file.txt  
git diff test_file.txt
```

- + added lines; - removed lines (in terminal colors when available).

- Stage and commit when the diff matches **one** clear idea:

- Stage and commit when the diff matches **one** clear idea:

- Stage and commit when the diff matches **one** clear idea:

```
git add test_file.txt
```

```
git commit -m "add second line to test file"
```

```
git log
```

- Paper: hard to **search**, easy to **lose**, handwriting fades.

- Paper: hard to **search**, easy to **lose**, handwriting fades.
- Git: **searchable** log, **next to code**, each step tied to a **message**.

- Paper: hard to **search**, easy to **lose**, handwriting fades.
- Git: **searchable** log, **next to code**, each step tied to a **message**.
- Answers **what**, **when**, and **why** without a parallel prose log.

`<type>: <short summary>`

`<optional longer explanation>`

## A commit-message shape

`<type>: <short summary>`

`<optional longer explanation>`

- Typical types: feature, bugfix, refactor, doc, test, cleanup.

`<type>: <short summary>`

`<optional longer explanation>`

- Typical types: feature, bugfix, refactor, doc, test, cleanup.
- Keep the first line short (often under ~72 characters).

- Write like a **command**: “add test for edge case,” not “added test.”

- Write like a **command**: “add test for edge case,” not “added test.”
- State **intent** and **problem solved**; the diff shows the code.

- Write like a **command**: “add test for edge case,” not “added test.”
- State **intent** and **problem solved**; the diff shows the code.
- Future you reads **git log** more than the old notebook margin.

- One **coherent** change set per commit.

- One **coherent** change set per commit.
- Avoid “everything from Tuesday” in a single commit.

- One **coherent** change set per commit.
- Avoid “everything from Tuesday” in a single commit.
- Avoid splitting one logical fix across many commits that only make sense together.

- `git add .` / `git add -A` grab all files in scope—often too much.

- `git add .` / `git add -A` grab all files in scope—often too much.
- Risk: secrets, local config, unrelated edits; muddy messages; hard reverts.

- `git add .` / `git add -A` grab all files in scope—often too much.
- Risk: secrets, local config, unrelated edits; muddy messages; hard reverts.
- Prefer `git add <path>...` explicitly.

- `git add .` / `git add -A` grab all files in scope—often too much.
- Risk: secrets, local config, unrelated edits; muddy messages; hard **reverts**.
- Prefer `git add <path>...` explicitly.
- `git add -u` updates tracked files only—still match the message.

- Name patterns git should **never** track.

- Name patterns git should **never** track.
- Typical entries: build artifacts, **.venv**, credentials, **\_\_pycache\_\_**, OS junk.

- Name patterns git should **never** track.
- Typical entries: build artifacts, **.venv**, credentials, **\_\_pycache\_\_**, OS junk.
- Keeps **git status** readable and avoids accidental leaks.

- Stage deliberately; commit with **scoped, imperative** messages.

- Stage deliberately; **commit** with **scoped, imperative** messages.
- **git log** + **git diff** = searchable lab record beside the code.

- Stage deliberately; **commit** with **scoped, imperative** messages.
- **git log** + **git diff** = searchable lab record beside the code.
- Next: Branching and workflows for branches, tags, and collaboration patterns.

- Russell A. Poldrack, [Better Code, Better Science](#) – book home (Essential tools and techniques, Version control).

- Russell A. Poldrack, [Better Code, Better Science](#) – book home (Essential tools and techniques, Version control).
- The curious coder's guide to git

- Russell A. Poldrack, [Better Code, Better Science](#) – book home (Essential tools and techniques, Version control).
- The curious coder's guide to git
- Git tutorial · Git cheat sheet (PDF) · Ubuntu CLI cheat sheet (PDF)